Actividad 6: Gestión de Excepciones Biblioteca

1. Objetivo

Al terminar la actividad debes ser capaz de:

- Crear una jerarquía de excepciones personalizadas con distintos parámetros en el constructor.
- Lanzar excepciones (throw new ...) desde la capa de servicio cuando se incumple una regla de negocio.
- Tratar todos los errores en un único punto con **@RestControllerAdvice** y **@ExceptionHandler**.
- Devolver siempre errores con el mismo formato JSON y el código HTTP adecuado.

2. Tareas

2.1. Excepción base

En el paquete **exception** crea la clase abstracta **BibliotecaException**, que extiende **RuntimeException** y tiene:

- String **codigo** (por ejemplo "LIBRO_NO_ENCONTRADO") con su getter.
- HttpStatus **status** con su getter.
- Un constructor (String codigo, HttpStatus status, String mensaje) que llame a super(mensaje).

2.2. Las 7 excepciones

Todas heredan de `BibliotecaException`. Cada una recibe los parámetros indicados y construye con ellos un **mensaje legible** que se pasa al constructor padre.

Excepción	Parámetros del constructor	HTTP	Código y ejemplo de mensaje
LibroNoEncontradoException	Long id	404	LIBRO_NO_ENCONTRADO No existe ningún libro con id 99
SocioNoEncontradoException	Long id	404	SOCIO_NO_ENCONTRADO No existe ningún socio con id 99
IsbnDuplicadoException	String isbn	409	ISBN_DUPLICADO Ya existe un libro con el ISBN 9780441172719
LibroNoDisponibleException	String titulo, int ejemplaresDisponibles	409	LIBRO_NO_DISPONIBLE El libro 'Neuromante' no tiene ejemplares disponibles (disponibles: 0)
LimitePrestamosExcedidoException	Long socioId, int limiteMaximo	409	LIMITE_PRESTAMOS_EXCEDIDO El socio 2 ha alcanzado el límite de 3 préstamos activos
SocioSancionadoException	Long socioId, LocalDate fechaFinSancion	403	SOCIO_SANCIONADO El socio 3 está sancionado hasta el 2026-11-18
DevolucionInvalidaException	Long prestamoId, String motivo	400	DEVOLUCION_INVALIDA No se puede devolver el préstamo 1: el préstamo ya fue devuelto

Pista. Así quedaría una excepción de otro dominio (no es una de las tuyas):

```
public class ProductoNoEncontradoException extends TiendaException {

    public ProductoNoEncontradoException(Long id) {
        super("PRODUCTO_NO_ENCONTRADO",
              HttpStatus.NOT_FOUND,
              "No existe ningún producto con id " + id);
    }
}
```

2.3. Lanzar las excepciones en los servicios

Abre los servicios y busca los comentarios `// TODO` (hay 10). Sustituye cada uno por el `throw new ...` correspondiente. **Los controladores no deben tener ningún try/catch.**

TODO	Método	Condición	Excepción
1	LibroService.crear	El ISBN ya existe	IsbnDuplicado
2	LibroService.buscarPorId	El libro no existe	LibroNoEncontrado
3	SocioService.buscarPorId	El socio no existe	SocioNoEncontrado
4	PrestamoService.prestar	El socio no existe	SocioNoEncontrado
5	PrestamoService.prestar	El libro no existe	LibroNoEncontrado
6	PrestamoService.prestar	Socio sancionado	SocioSancionado
7	PrestamoService.prestar	Socio con 3 o más préstamos	LimitePrestamosExcedido
8	PrestamoService.prestar	Libro sin ejemplares	LibroNoDisponible
9	PrestamoService.devolver	El préstamo no existe	DevolucionInvalida
10	PrestamoService.devolver	El préstamo ya está devuelto	DevolucionInvalida

Importante: el orden de las comprobaciones en `prestar` es una regla de negocio (socio existe → libro existe → no sancionado → límite → ejemplares) y ya está respetado en el código. No lo cambies.

Mientras no lances las excepciones, los casos de error provocarán un `NoSuchElementException` o un resultado incorrecto: es normal. Fíjate también en que `Optional.get()` no es la mejor forma de tratar un vacío: más adelante puedes refactorizarlo con `orElseThrow(() -> new ...)`.

2.4. Tarea 4. Manejo centralizado de errores

Crea la clase `ErrorResponse` y la clase `GlobalExceptionHandler` anotada con `@RestControllerAdvice`. Todas las respuestas de error deben tener esta estructura:

```
{
  "timestamp": "2026-10-19T10:15:30",
  "status": 409,
  "codigo": "LIBRO_NO_DISPONIBLE",
  "mensaje": "El libro 'Neuromante' no tiene ejemplares disponibles (disponibles: 0)",
  "path": "/api/prestamos"
}
```

Implementa estos cuatro manejadores:

Excepción tratada	HTTP	Comportamiento
BibliotecaException	el de la excepción	Usa su status y su codigo. Cubre las 7 excepciones con un solo método.
MethodArgumentNotValidException	400	codigo VALIDACION_ERROR. Rellena detalles con un texto por campo erróneo (campo: mensaje).
HttpMessageNotReadableException	400	codigo JSON_INVALIDO. Mensaje genérico indicando que el cuerpo no es un JSON válido.
Exception	500	codigo ERROR_INTERNO. Mensaje genérico. No expongas la traza ni el mensaje original al cliente.

Para obtener el `path` puedes recibir un parámetro `HttpServletRequest request` en el método del handler y usar `request.getRequestURI()`.